Language-Agnostic Dependency Visualizer

Productivity Documentation & Implementation Roadmap

Developer Productivity Features

Universal Language Support (Agnostic Engine)

Detects and parses Go, Node.js, Python, and Java projects automatically. Developers don't need separate tools for different stacks.

Interactive "Drill-Down" Visualization

Interactive D3.js web interface allows zooming into specific sub-graphs. Prevents cognitive overload in massive microservice architectures.

Version Conflict & Vulnerability Overlay

Directly highlights nodes with security risks or multiple version resolution issues, saving hours of manual debugging.

Dependency Diffing

Compare two branches or commits to see a visual "diff" of what dependencies were added/removed. Essential for safe PR reviews.

Blast Radius Analysis

Click on a single library to see every internal module that depends on it. Helps estimate the impact of a version upgrade.

Implementation Roadmap

Phase	Core Milestones	Productivity Outcome
Phase 1: Foundation	• Internal Graph Data Model (JSON Schema) • Go Module Provider (go mod graph integration) • Static DOT/Mermaid export	Instant visibility into Go projects; replaces manual dependency hunting with a single command.
Phase 2: Agnostic Layer	• Provider Interface architecture • NPM/Yarn Provider implementation • Python (Pip/Poetry) Provider implementation	Eliminates context switching for polyglot developers; provides a unified tool for the whole stack.
Phase 3: Interactive UI	• Local Web Server (Go net/http) • D3.js Force-Directed Graph • Search, Filter, and Depth control toggles	Navigational speed; developers can find specific nodes in seconds rather than scrolling through text logs.
Phase 4: Intelligence	• Integration with OSV/Govulncheck API • Version conflict detection logic • License compliance auditing	Proactive maintenance; identifies technical debt and security risks during the development phase.
Phase 5: Workflow	• Git-Diff integration • CI/CD Action (GitHub Action/GitLab CI) • Automated Documentation Generation	Safety at scale; prevents bloated or risky dependencies from entering the main branch automatically.